

AI Development Standards

A practical standard for safe, consistent AI-assisted engineering

AI Engineering Starter Kit | Current standard | July 2026

Operating model

The repository is durable AI context; skills are repeatable procedures; automated gates provide evidence; accountable people publish changes.

- AI reads AGENTS.md and applicable .ai/* .md before work.
- AI output remains untrusted until inspected, tested and approved.
- AI never stages, commits, pushes, changes branches, rewrites history or opens/merges pull requests.
- A human uses the IDE or approved manual Git tooling for every publication action.

Context engineering

Context file	Purpose
.ai/project.md	Language, framework, runtime and project-specific constraints
.ai/architecture.md	Layers, dependency direction and boundary rules
.ai/coding-standards.md	Readability, naming, helpers, interfaces and implementation rules
.ai/testing.md	Frameworks, mocks, assertions, coverage and integration tests
.ai/dependencies.md	Approved libraries and tools
.ai/tooling-policy.md	Decision process when a required tool is unspecified
.ai/version-control-policy.md	Human-only staging, commit, push and PR publication

Language profiles

Generate a lean product repository from one profile. Do not clone the complete catalogue into every product.

Profile	Maturity	Included verification
Java / Spring Boot	Runnable	Maven, JUnit 5, Mockito, AssertJ, Testcontainers
Python / FastAPI	Runnable	Pydantic v2, Ruff, mypy, pytest
C# / ASP.NET Core	Runnable	.NET 9, xUnit, NSubstitute, FluentAssertions
C, C++, Rust, Scala	Scaffold	Standards context; product adds build/test payload
Node, JavaScript, TypeScript	Scaffold	Standards context; product adds build/test payload
React / Next.js TypeScript	Scaffold	Frontend context; product adds UI template and gates

Tool and library selection

- Use the library or tool recorded in the active language context and manifests.
- If no approved choice exists, stop and ask the user before adding a dependency.
- Explain need, alternatives, licence, provenance, security, compatibility and maintenance cost.
- Ask whether approval is task-local or an ongoing project standard.
- For an ongoing standard, update AI context, manifest, lockfile, CI and hooks together.

Readable, testable design

Technique	Use	Avoid
Interface	External or genuinely substitutable boundary	One interface per concrete class
Focused collaborator	Named mapper, policy, validator or normalizer	Generic Helper, Utils or Manager
Pure component	Explicit input/output transformation	Hidden mutable or global state
Mock	External, slow or non-deterministic collaborator	Mocking every internal object

Prefer dependency injection, precise responsibilities and observable-behaviour tests. Excessive mock setup is a design warning.

Testing standard

- Every test contains at least one meaningful outcome assertion.
- Interaction verification alone is insufficient.
- Mock external collaborators, not value objects or the unit's internal implementation.
- Use real integration boundaries where risk justifies them.
- Python uses pytest and strict Pydantic models; constrain mutable module-level state.
- Java uses JUnit 5, Mockito and AssertJ; C# uses xUnit, NSubstitute and FluentAssertions.

Reusable skill workflow

Stage	Skill family	Evidence
Implement	create-spring/python/dotnet-feature	Changed files, tests and validation
Test	generate-java/python/dotnet-tests	Behavioural assertions and targeted results
Review	review-java/python/dotnet-change	Severity-ranked findings with file/line evidence
Secure	secure-java/python/dotnet-review	Threat-focused findings and remediation
Diagnose / refactor	Java specialist skills	Root-cause or behaviour-preservation evidence

Quality gates and CI

- The catalogue CI validates all profiles, then generates and independently verifies Java, Python and C#.
- Java: Spotless, Checkstyle, tests, coverage and Testcontainers.
- Python: Ruff format/lint, strict mypy and pytest.
- C#: dotnet format, warnings-as-errors build and tests.
- CI repeats local gates and does not grant AI authority to publish code.

Quick start

```
scripts\create-project.ps1 -Profile python `
-ProjectName payments-api `
-Destination C:\Workspace\payments-api
```

- Define product-specific rules and domain language.
- Record owners, classification, external systems and deployment constraints.
- Confirm the verification baseline runs locally and in CI.
- Install only the matching skills.
- Human reviews, stages, commits and pushes.

Definition of done

- Relevant context was read and remains internally consistent.
- Approved tools and libraries were used.
- Architecture and API contracts are respected.
- Tests prove positive, negative and boundary behaviour.
- Formatting, static analysis, security and tests pass.
- Residual risks and unverified areas are explicit.
- An accountable human performs version-control publication and approval.